Hands-On Lab Walkthrough: PySpark Batch ETL & DataFrames

Course: Enterprise Real-Time Data Engineering

Session: S08 — Apache Spark Architecture & PySpark DataFrames

Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)

Materials: fakhr.me/KafkaSpark | Repo: github.com/fakhruldeen/KafkaSpark

1. Lab Architecture & Pedagogical Objectives

In this lab, you will build and execute an enterprise-grade batch ETL pipeline using **Apache Spark (PySpark)** for the **Nile Retail Group**:

1. Configure and inspect the PySpark execution environment and JVM connectivity.
2. Formulate explicit data schemas (`StructType`) avoiding the pitfalls of runtime type inference.
3. Apply data quality validation rules to separate valid transactions from poisoned rows.
4. Eliminate duplicated events using distributed deduplication (`dropDuplicates`).
5. Write curated analytical datasets into **Apache Parquet** partitioned by store (`partitionBy("store_id")`), while routing defective records to an isolated quarantine folder.
6. Verify pipeline correctness using the automated pytest test suite (`test_s08_batch.py`).

2. Prerequisites & Environment Setup

Open your terminal and navigate to the lab root directory:

```
cd delivery_materials/00_Lab 2>/dev/null || cd ../../00_Lab || cd 00_Lab
```

Step 2.1: Verify Java 17 and Python Environment

Ensure Java 17 Temurin and the PySpark virtual environment are correctly sourced:

```
source .venv/bin/activate
echo "Java Version:"
java -version
echo "Python Version:"
python3 --version
```

Expected Output:

```
openjdk version "17.0.21" 2026-10-20
OpenJDK Runtime Environment Temurin-17.0.21+9
Python 3.12.x
```

Step 2.2: Verify PySpark Import & Cluster Master

Run a one-line PySpark probe:

```
python3 -c "
from pyspark.sql import SparkSession
spark = SparkSession.builder.master('local[1]').appName('probe').getOrCreate()
print('PySpark Version:', spark.version)
spark.stop()
"
```

Expected Output:

```
PySpark Version: 3.5.9
```

---

3. Part 1: Automated Unit Testing (`test_s08_batch.py`)

Before running against production files, execute the unit test suite verifying each business rule against isolated in-memory datasets.

Step 3.1: Inspect the Test Suite

Open and read `tests/test_s08_batch.py` :

```
cat tests/test_s08_batch.py
```

Notice the five core assertions:

1. `test_row_with_missing_or_non_positive_amount_is_quarantined` : Verifies missing and negative amounts are intercepted.
2. `test_unknown_event_type_is_quarantined` : Rejects invalid event types (e.g., `'Shipped'`).
3. `test_duplicate_event_id_is_kept_once` : Validates deduplication by `event_id` .
4. `test_counts_always_add_up_to_the_input` : Confirms conservation of records ($Read = Clean +$

Quarantined + Duplicates).

5. `test_clean_output_is_partitioned_by_store`: Checks the creation of `store_id=*` directory partitions and `_SUCCESS` markers.

Step 3.2: Execute the Pytest Suite

Run the unit tests:

```
pytest -v tests/test_s08_batch.py
```

Expected Output:

```
tests/test_s08_batch.py::test_row_with_missing_or_non_positive_amount_is_quarantined PASSED
[ 20%]
tests/test_s08_batch.py::test_unknown_event_type_is_quarantined PASSED                    [
40%]
tests/test_s08_batch.py::test_duplicate_event_id_is_kept_once PASSED                      [
60%]
tests/test_s08_batch.py::test_counts_always_add_up_to_the_input PASSED                    [
80%]
tests/test_s08_batch.py::test_clean_output_is_partitioned_by_store PASSED                 [1
00%]

============================== 5 passed in 10.45s ==============================
```

---

4. Part 2: Inspecting the Batch ETL Implementation (`python/s08_batch.py`)

Open `python/s08_batch.py` to examine the production engineering logic:

```
cat python/s08_batch.py
```

Core Implementation Walkthrough:

1. **Explicit Schema (`RAW_SCHEMA`):**

```
python
   RAW_SCHEMA = StructType([
        StructField("event_id", StringType(), True),
        StructField("order_id", StringType(), True),
        StructField("customer_id", StringType(), True),
        StructField("store_id", StringType(), True),
        StructField("event_type", StringType(), True),
        StructField("amount", DoubleType(), True),
```

```
        StructField("event_time", StringType(), True),
    ])
```

2. **Data Quality Rules Engine:**

```
python
    valid = (F.col("event_id").isNotNull()
             & F.col("order_id").isNotNull()
             & F.col("event_type").isin("OrderPlaced", "PaymentAccepted", "Refunded")
             & F.col("amount").isNotNull() & (F.col("amount") > 0))
```

3. **Partitioned Parquet Sink:**

```
python
    clean.write.mode("overwrite").partitionBy("store_id").parquet(f"{output_path}/orders")
    quarantine.write.mode("overwrite").parquet(f"{output_path}/quarantine")
```

---

5. Part 3: Running the Full Pipeline on Nile Retail Transactions

Step 5.1: Execute the Batch Job

Process the raw transaction log in `data/orders.jsonl`:

```
python3 -m python.s08_batch --input data/orders.jsonl --output data/curated
```

Expected Output:

```
{'read': 4158, 'clean': 4002, 'quarantined': 23, 'duplicates_removed': 133}
```

Step 5.2: Inspect Partitioned Orders

Examine the curated orders directory:

```
ls -la data/curated/orders/
```

Expected Structure:

```
_SUCCESS
store_id=ALX-01/
store_id=ALX-02/
store_id=CAI-01/
store_id=CAI-02/
store_id=GIZ-01/
```

Step 5.3: Inspect Quarantined Defective Records

Check the quarantined records:

```
ls -la data/curated/quarantine/
```

Notice the presence of Snappy-compressed Parquet files containing exclusively invalid transactions (negative amounts, missing event IDs, or corrupt event types).

---

6. Part 4: Interactive Inspection via PySpark Shell

Launch the interactive PySpark shell to inspect the generated Parquet dataset:

```
python3 -c "
from pyspark.sql import SparkSession
spark = SparkSession.builder.master('local[1]').appName('inspect').getOrCreate()
df = spark.read.parquet('data/curated/orders')
print('=== Clean Dataset Schema ===')
df.printSchema()
print('=== Sample Rows ===')
df.select('order_id', 'store_id', 'amount', 'event_ts').show(5, truncate=False)
spark.stop()
"
```

Expected Output:

```
=== Clean Dataset Schema ===
root
 |-- event_id: string (nullable = true)
 |-- order_id: string (nullable = true)
 |-- customer_id: string (nullable = true)
 |-- event_type: string (nullable = true)
 |-- amount: double (nullable = true)
 |-- event_time: string (nullable = true)
 |-- event_ts: timestamp (nullable = true)
 |-- store_id: string (nullable = true)

=== Sample Rows ===
+--------+--------+------+-------------------+
|order_id|store_id|amount|event_ts           |
+--------+--------+------+-------------------+
|ord-0001|CAI-01  |150.0 |2026-11-06 00:00:00|
|ord-0002|CAI-01  |220.5 |2026-11-06 00:05:00|
+--------+--------+------+-------------------+
```

---

7. Automated Gate Verification

Run the comprehensive S08 automated verification suite to validate all 5 Quality Gates:

```
bash ../S08_Apache_Spark_and_PySpark_DataFrames/Trainer/verify_s08.sh
```

Expected Output:

```
Running S08 Automated Apache Spark & PySpark Verification Gate...
[1/5] Probing Java 17 runtime & PySpark installation... PASS (Java 17 Temurin active, PySpar
k 3.5.9 loaded)
[2/5] Initializing SparkSession and Catalyst Engine... PASS (SparkSession created, Catalyst
in-memory execution verified)
[3/5] Running S08 PySpark batch test suite (pytest)... PASS (5/5 tests passed: Quarantine, S
chema, Deduplication, Balance, Partitioning)
[4/5] Executing full batch ETL on data/orders.jsonl... PASS (Clean records: 4002, Quarantine
d: 23)
[5/5] Inspecting partitioned Parquet structure and quarantine folder... PASS (_SUCCESS prese
nt, 6 store partitions found, quarantine verified)

✔ S08 Apache Spark & PySpark DataFrames Quality Gate: 100% PASS
```

🌟 Gold Stretch Challenge: Off-Heap Tungsten Execution & Physical Plan Disassembly

> **Distinguished Engineer Challenge:** If you complete the core lab before time expires, analyze whole-stage code generation:
> 1. Generate an analytical DataFrame with 1,000,000 synthetic transactions:
> `python`
> `df = spark.range(0, 1000000).withColumn("category", (F.col("id") % 10).cast("string"))`
> `result = df.groupBy("category").agg(F.count("id").alias("cnt"), F.sum("id").alias("total"))`
> 2. Inspect the formatted physical plan using `result.explain(mode="formatted")`.
> 3. Identify the `*` asterisks designating WholeStageCodegen operators, and note how HashAggregate compiles directly into Java byte code avoiding JVM object instantiation.